

From Vibe Coding to Agentic Engineering

Andrej Karpathy — Fireside Chat, Sequoia AI Ascent 2026 • April 30, 2026

Original blog summary: karpathy.bearblog.dev/sequoia-ascent-2026/

This document is an archival summary of Andrej Karpathy's fireside chat at Sequoia Capital's AI Ascent 2026 conference (April 30, 2026) and his corresponding blog post. Content is reconstructed from the original blog post and contemporaneous reporting. Direct quotations are sourced from Karpathy's X/Twitter thread (@karpathy, May 2026) and verified secondary coverage.

Overview

At Sequoia AI Ascent 2026, Andrej Karpathy argued that we are in the middle of a paradigm shift in software development — not merely an acceleration of what existed before. The central thesis: LLMs represent a genuinely new type of computer, and the role of human programmers is evolving from writing code to directing intelligent agents. He introduced the term **Agentic Engineering** to distinguish professional, quality-preserving AI-assisted development from the looser "vibe coding" that characterized early 2025.

Karpathy described December 2025 as the inflection point when agentic coding shifted from experimental to reliable: programmers who once corrected individual lines began delegating entire subsystem refactors. The talk opens a new conceptual vocabulary for the era now underway.

The Three Software Eras

Era	Paradigm	What humans do	Interpreter
Software 1.0	Explicit rules & algorithms	Write code in traditional languages (C++, Python, Java)	Computer / CPU
Software 2.0	Learned weights (neural networks)	Curate datasets, define objectives, train models	Neural network

Software 3.0	LLM-based programming via natural language	Write prompts, design specs, curate context, evaluate outputs	LLM over context window
---------------------	--	---	-------------------------

In Software 3.0, **the context window becomes the new program**. You populate it with prompts, documentation, examples, tool definitions, memory, and instructions — and the LLM interprets it, performing computation over information. This is a meaningful inversion: the "program" is now text rather than compiled binary, and the "interpreter" is a language model rather than a CPU.

LLMs as a New Type of Computer

The LLM Computer Analogy

The LLM OS has no traditional operating system in the middle. Instead:

- **Context window = RAM**
— working memory that holds your current program and state
- **Model weights = CPU**
— the fixed hardware doing computation
- **Prompts and context = software**
— what you write to make the machine do things

This reframing has significant implications: "programming" an LLM is now closer to configuring hardware than writing traditional software.

Karpathy's core provocation: *AI is not just a faster way to build the old apps*. The correct question is not "how do we use AI to speed up our existing software?" but "what new categories of software become possible — and what existing apps should stop existing as apps at all?"

New Horizons: Three Examples Karpathy Highlighted

1. **menugen** — An app that generates a restaurant menu from raw ingredients and customer context. In Software 1.0 terms, this would be a complex rules engine. In Software 3.0, the "app" largely disappears: the neural network directly transforms input (inventory data,

customer profile) into output (personalized menu). Much of the traditional application layer becomes unnecessary.

2. **Installation as agent task** — Installation flows, traditionally a brittle sequence of user interactions, become a block of instructions you paste into an agent. The agent reads the local environment, debugs errors, adapts to the machine, and completes setup. The human writes the spec; the agent executes and recovers from failures.
3. **Autonomous subsystem refactors** — By December 2025, coding agents could reliably take a high-level refactoring objective ("extract this service into its own module with these contracts") and execute it across a codebase, with a human reviewing the resulting diff rather than writing individual lines.

Vibe Coding vs. Agentic Engineering

Karpathy coined "vibe coding" in early 2025 to describe a casual, low-rigor style of AI-assisted programming: the programmer prompts loosely, accepts outputs without deep inspection, and tolerates a degree of correctness ambiguity. Vibe coding is fast and democratizing — it raises the floor for non-programmers.

Agentic Engineering is the professional discipline that replaces it at the ceiling. It is not looser vibe coding applied to larger tasks. It requires new rigor:

What the Agentic Engineer Does

- **Spec design**
 - Writing a detailed specification before prompting: documentation, invariants, security boundaries, edge case handling. Not just "plan mode," but a formal artifact the agent is held accountable to.
- **Supervision of agent plans**
 - Reviewing and correcting the agent's approach before execution, not just its output after.
- **Diff inspection**
 - Reading and critically evaluating every change the agent makes, not rubber-stamping output.
- **Evaluation loop design**

— Writing tests and evaluations so the agent can self-check and iterate. The human's job shifts from writing code to writing the tests that validate code.

- **Permission management and isolation**

— Managing what agents can access, running them in isolated worktrees to limit blast radius of mistakes.

- **Quality preservation**

— Maintaining correctness, security, maintainability standards that an agent optimizing for local task completion might degrade.

"Agentic engineering is the disciplined practice of coordinating AI coding agents to ship professional software at scale while preserving security, maintainability, and the quality bar."

The key differentiator: *agentic engineering extrapolates the ceiling, while vibe coding raises the floor*. People who are very good at agentic engineering can achieve productivity multiples far beyond the traditional "10x engineer" — not because the agent is perfectly reliable, but because the human's judgment about where to invest attention, what to delegate, and what to verify becomes the scarce resource.

Jagged Intelligence

Karpathy extended the "jagged frontier" concept (from Mollick's BCG study) with his own framing: **LLMs are "jagged ghosts."** Models spike dramatically in domains with dense training signal — mathematics, code with tests, games with scores — and fail unexpectedly in tasks that seem simpler. The pattern is jagged because it doesn't correlate with human intuitions about difficulty.

The practical implication for agentic engineering: you cannot predict where an agent will succeed without hands-on experience. Evaluation loops and testing are essential precisely because the reliability profile is unintuitive. An agent that reliably refactors a 2,000-line class may hallucinate a non-existent API in a one-liner.

The December 2025 Inflection Point

Karpathy named December 2025 as the moment when agentic coding became genuinely reliable rather than experimental. He specifically cited Claude Code (Anthropic) and equivalent tools as the examples that crossed this threshold. The shift: agents could maintain context across multi-file changes, use tools (file system, terminals, test runners) reliably, and loop through error-correction until tasks completed.

Before this inflection point, programmers used AI primarily as a sophisticated autocomplete — suggesting lines or functions within their direct supervision. After it, the delegation unit became *subsystems and features*, with the human reviewing outcomes rather than supervising execution step by step.

Implications for Organizations

1. **The agentic engineer role is the highest-leverage new hire** — someone who understands how to design specs, structure agent workflows, write evaluations, and maintain quality. This is not a junior role; it requires deep software intuition applied to a new medium.
2. **Apps should be questioned, not just optimized** — Organizations deploying AI should ask not only "how do we use AI in our existing app?" but "does this app need to exist in its current form?" Many application layers become unnecessary when LLMs directly transform inputs to outputs.
3. **The context window is the competitive moat** — What you put in the context window — institutional knowledge, high-quality examples, precise specifications, domain constraints — is the new source of advantage. Generic prompting produces generic results; deep context engineering produces proprietary capability.
4. **Evaluation is the new programming skill** — Writing tests and evaluations that allow agents to self-check is becoming as important as writing the code itself. Organizations should invest in evaluation infrastructure, not just in deployment of agents.
5. **Speed of skill development compounds** — Individuals and organizations that invest in hands-on agentic engineering now build intuition about the jagged capability profile that cannot be acquired from training or documentation. The compounding advantage is real and already visible.

Relevance to Earlier Frameworks

Karpathy's Software 3.0 thesis directly extends his 2017 "Software 2.0" essay (Medium), where he argued that neural networks were replacing traditional code in narrow domains. Software 3.0 generalizes further: LLMs reduce the need for domain-specific neural network training by enabling general-purpose computation over natural language context.

The talk also connects to Karpathy's broader view that LLMs represent a new substrate for computation — not a feature added to existing computers, but a genuinely different class of machine with different strengths, failure modes, and programming interfaces.

Andrej Karpathy is an AI researcher and entrepreneur who was a founding member of OpenAI, Director of AI at Tesla, and is currently an independent researcher and investor. This document archives his fireside chat at Sequoia AI Ascent 2026 (April 30, 2026). The original blog summary is at karpathy.bearblog.dev/sequoia-ascent-2026/.